NLU Assignment-02

Sangini Garg (B22CS045)

24/03/2026

PROBLEM 1: LEARNING WORD EMBEDDINGS FROM IIT JODHPUR DATA

TASK 1: DATASET PREPARATION

1. Data Collection

Textual data was collected from multiple IIT Jodhpur sources including academic regulations, curriculum documents, and institute-related content.

Some files were labeled as `.txt` but were actually PDF files internally. When read as plain text, they produced unreadable content such as "endobj" and "flatedecode". To handle this, files were checked for the PDF signature (`%PDF`). If detected, text was extracted using a PDF parser; otherwise, files were read normally.

2. Preprocessing

The raw text was cleaned using the following steps:

- Removal of non-ASCII characters
- Removal of URLs and email addresses
- Removal of boilerplate content (copyright lines, navigation text)
- Conversion to lowercase
- Removal of numbers and punctuation
- Tokenization into words
- Removal of stopwords (e.g., "the", "is")
- Removal of very short tokens (length < 3)
- Removal of domain-specific noise words (e.g., iit, jodhpur)

3. Dataset Statistics

- Total documents: 5
- Total tokens: 16457
- Vocabulary size: 2379

The vocabulary size is significantly smaller than total tokens, indicating repetition and consistency in language, which is beneficial for training.

```
==================================================
DATASET STATISTICS
==================================================
  Total documents      : 5
  Total tokens         : 16457
  Vocabulary size      : 2379

  Top 20 most frequent words:
    student                    -> 335
    course                     -> 278
    degree                     -> 182
    semester                   -> 176
    students                   -> 166
    phd                        -> 164
    mtech                      -> 145
    program                    -> 145
    courses                    -> 140
    academic                   -> 136
    requirements               -> 132
    grade                      -> 130
    credits                    -> 128
    minimum                    -> 126
    engineering                -> 125
    science                    -> 120
    may                        -> 107
    must                       -> 102
    dual                       -> 95
    examination                -> 94
```

Figure 1: Dataset statistics

4. Cleaned Corpus

The processed tokens were stored in a text file where each document occupies one line and words are space-separated. This format is suitable for Word2Vec training.

5. Word Cloud

A word cloud was generated to visualize frequent words. Words such as *student*, *course*, and *degree* appeared prominently, indicating correct preprocessing.

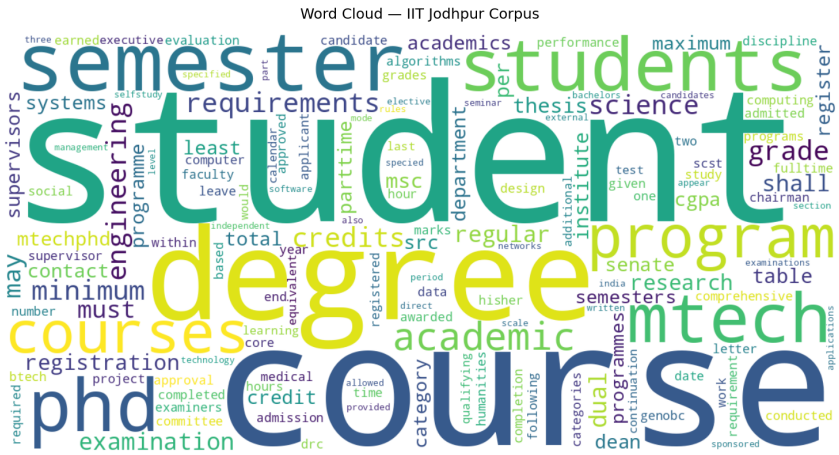

Figure 2: Word cloud showing most frequent words in the corpus

TASK 2: MODEL TRAINING

1. Objective

Two Word2Vec models were trained:

- Continuous Bag of Words (CBOW)
- Skip-gram with Negative Sampling

2. Training Setup

Both models were trained using:

- Vector size: 100
- Window size: 5
- Negative samples: 5

3. Experiments

A. Embedding Dimension:

- Tested 100 and 200 dimensions
- Higher dimensions provided more detailed representations

B. Context Window Size:

- Tested window sizes 5 and 10
- Larger window captured broader context but reduced precision

C. Negative Sampling:

- Tested 5 and 10 negative samples
- Higher negative sampling improved word separation

```
==================================================
EXPERIMENT RESULTS COMPARISON
==================================================

CBOW (vector_size=100, window=5, negative=5):
[('algorithms', 0.9993537068367004), ('systems', 0.9993152618408203), ('computing', 0.9992377161979675), ('data',
0.9991783499717712), ('requirements', 0.9991114139556885), ('may', 0.9991019368171692), ('core',
0.9990963935852051), ('design', 0.9990174174308777), ('shall', 0.9989479780197144), ('courses', 0.9989339709281921)]

CBOW (vector_size=200):
[('systems', 0.9997864961624146), ('core', 0.9997838735580444), ('algorithms', 0.999772846698761), ('computing',
0.9997685551643372), ('course', 0.999759316444397), ('semester', 0.9997580647468567), ('data', 0.9997466206550598),
('academic', 0.9997444748878479), ('may', 0.9997372627258301), ('design', 0.9997356534004211)]

Skip-gram (window=5, negative=5):
[('allowed', 0.9952908754348755), ('appear', 0.9952709078788757), ('within', 0.9950435757637024), ('permitted',
0.994857132434845), ('shall', 0.9947840571403503), ('registration', 0.9947683811187744), ('registered',
0.9945809245109558), ('date', 0.9945712089538574), ('approved', 0.9945284724235535), ('courses',
0.9944833517074585)]

Skip-gram (window=10):
[('academic', 0.9821695685386658), ('registration', 0.9803915023803711), ('within', 0.9776270389556885), ('thesis',
0.9762237071990967), ('src', 0.9751889705657959), ('may', 0.9738765954971313), ('shall', 0.9738137125968933),
('register', 0.972737729549408), ('date', 0.9718408584594727), ('per', 0.970864474773407)]

Skip-gram (negative=10):
[('shall', 0.9951519966125488), ('approved', 0.9950950145721436), ('registration', 0.9949942827224731),
('committee', 0.9948396682739258), ('may', 0.9947474598884583), ('candidate', 0.9946249723434448),
('recommendation', 0.9945638179779053), ('src', 0.9945403933525085), ('academic', 0.9943476319313049), ('within',
0.9943086504936218)]
```

Figure 3: Experiment Results

4. Model Comparison

CBOW captured general semantic relationships, while Skip-gram captured more contextual usage patterns.

TASK 3: SEMANTIC ANALYSIS

1. Nearest Neighbors

Top 5 nearest neighbors were computed using cosine similarity.

Observations:

- CBOW produced more general associations
- Skip-gram produced more contextually meaningful words
- Some words like "exam" were not present in vocabulary

2. Analogy Experiments

Examples tested:

- UG : BTech :: PG : ?
- Student : Course :: Teacher : ?

- PhD : Research :: BTech : ?

Observations:

- Many analogies failed due to limited dataset
- Some partial relationships were observed

TASK 4: VISUALIZATION

Method

PCA was used to project word embeddings into 2D space.

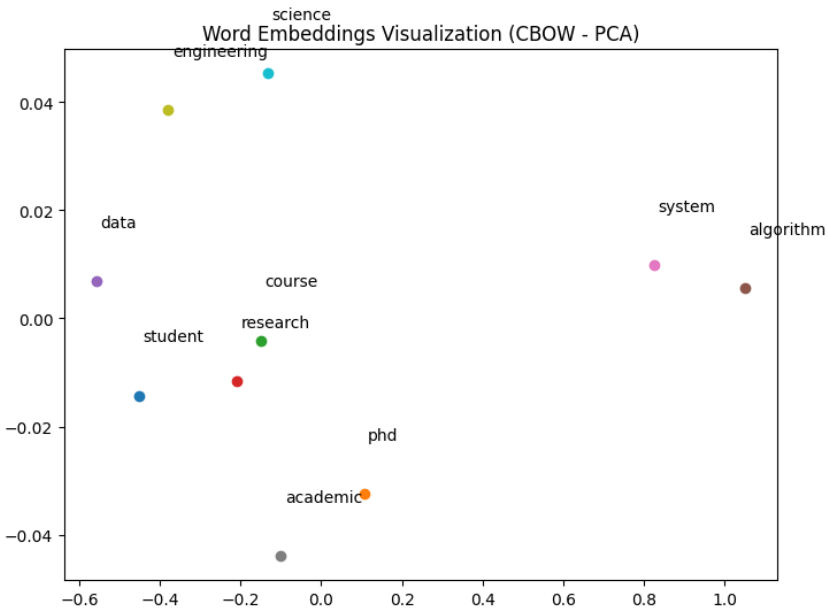

Figure 4:

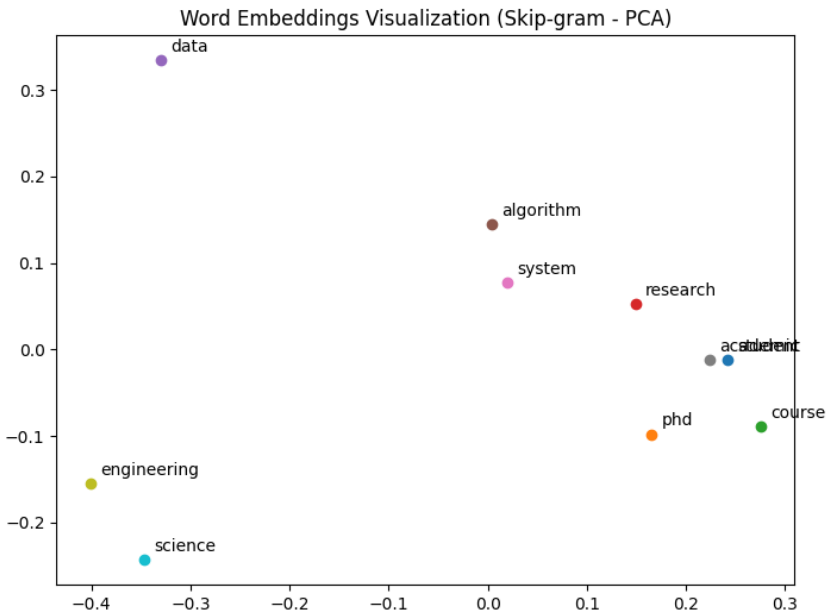

Figure 5:

Observations

- Both CBOW and Skip-gram showed clustering
- Academic terms formed clear groups (course, credits, semester)
- Research-related terms also clustered together (phd, research)
- Skip-gram produced more distinct clusters compared to CBOW

FINAL CONCLUSION

Word2Vec successfully captured semantic relationships from the dataset.

- CBOW performed better for general similarity
- Skip-gram performed better for contextual understanding

Limitations:

- Small dataset size
- Missing vocabulary
- Domain-specific bias

PROBLEM 2: CHARACTER-LEVEL NAME GENERATION USING RNN VARIANTS

The objective of this assignment is to design and compare different recurrent neural network architectures for character-level name generation. The models aim to learn patterns in Indian names and generate new names that are realistic, diverse, and novel.

TASK 0: DATASET

A dataset of 1000 Indian-like names was generated programmatically using combinations of common prefixes and suffixes.

Dataset Creation

Names were created by randomly combining:

- Prefixes (e.g., ra, su, vi, ka)
- Suffixes (e.g., esh, an, ita, dev)

Each name was capitalized and stored in `TrainingNames.txt`.

Preprocessing

- Duplicate names were removed
- Unique names were stored for evaluation
- Character vocabulary was constructed

Each name was converted into character-level sequences, where the model learns transitions from one character to the next.

TASK 1: MODEL IMPLEMENTATION

Three models were implemented from scratch:

1. Vanilla RNN

Architecture:

- One-hot character input
- Hidden state updated using tanh activation
- Output layer predicts next character

Hyperparameters:

- Hidden size: 64
- Learning rate: 0.01
- Epochs: 20

Parameter Count: Reported during execution

Key Idea: The model learns sequential dependencies by updating a hidden state at each character step.

```
Trainable Parameters: 8288
epoch 0 loss 4239.228045582771
epoch 1 loss 2948.6130747795105
epoch 2 loss 2660.299067735672
epoch 3 loss 2539.541642308235
epoch 4 loss 2504.832003235817
epoch 5 loss 2545.4141821861267
epoch 6 loss 2473.7222216129303
epoch 7 loss 2473.267795920372
epoch 8 loss 2490.4662169218063
epoch 9 loss 2662.3185917139053
epoch 10 loss 2530.348492026329
epoch 11 loss 2534.210798084736
epoch 12 loss 2585.1482669711113
epoch 13 loss 2632.2340223789215
epoch 14 loss 2484.81591963768
epoch 15 loss 2627.5639360547066
epoch 16 loss 2529.39479637146
epoch 17 loss 2488.5931429862976
epoch 18 loss 2461.211664199829
epoch 19 loss 2545.2015268802643
```

Figure 6: Implementation results for Vanilla RNN

2. Bidirectional LSTM (BLSTM)

Architecture:

- Embedding layer
- Bidirectional LSTM (forward + backward context)
- Fully connected output layer

Hyperparameters:

- Hidden size: 64
- Learning rate: 0.01
- Epochs: 20

Parameter Count: Reported during execution

Key Idea: BLSTM captures both past and future context, improving sequence understanding compared to vanilla RNN.

```
Trainable Parameters: 72736
epoch 0 loss 254.50089059374295
epoch 1 loss 37.50020626792684
epoch 2 loss 32.90911034567398
epoch 3 loss 30.905360890494194
epoch 4 loss 27.813796564114455
epoch 5 loss 26.763481639944075
epoch 6 loss 26.3763095766044
epoch 7 loss 26.416038533938263
epoch 8 loss 27.382026484789094
epoch 9 loss 91.04357404975963
epoch 10 loss 109.85928744739795
epoch 11 loss 32.58210335856711
epoch 12 loss 29.27212998635514
epoch 13 loss 27.377242963579192
epoch 14 loss 26.517759415677574
epoch 15 loss 26.437374497683777
epoch 16 loss 25.84251469494484
epoch 17 loss 25.73505223969005
epoch 18 loss 25.508506035967002
epoch 19 loss 25.134314545623965
generated samples saved
```

Figure 7: Implementation results for BLSTM

3. RNN with Attention

Architecture:

- Embedding layer
- RNN layer
- Attention mechanism over all hidden states
- Combined context + last hidden state

Hyperparameters:

- Hidden size: 64
- Learning rate: 0.01
- Epochs: 20

Parameter Count: Reported during execution

Key Idea: Attention allows the model to focus on important parts of the sequence rather than relying only on the last hidden state.

```
Trainable Parameters: 19235
epoch 0 loss 9711.05512946341
epoch 1 loss 9549.510199307293
epoch 2 loss 9412.344408306899
epoch 3 loss 9135.677694435237
epoch 4 loss 9349.88037618648
epoch 5 loss 9504.398300955649
epoch 6 loss 9548.985256982907
epoch 7 loss 9797.31456625009
epoch 8 loss 9417.67481443212
epoch 9 loss 9498.846688587932
epoch 10 loss 9583.423104948324
epoch 11 loss 9483.977138976163
epoch 12 loss 9345.374266134775
epoch 13 loss 9608.828649100127
epoch 14 loss 9265.051976384459
epoch 15 loss 9156.933826941886
epoch 16 loss 9306.155956141607
epoch 17 loss 9250.377766261774
epoch 18 loss 9352.843485127232
epoch 19 loss 9375.759960968824
new attention samples saved
```

Figure 8: Implementation results for Attention RNN

TASK 2: QUANTITATIVE EVALUATION

Two metrics were used:

1. Novelty Rate

$$\text{Novelty Rate} = \frac{\text{Number of generated names not in training set}}{\text{Total generated names}}$$

Interpretation: Measures how creative the model is.

2. Diversity

$$\text{Diversity} = \frac{\text{Unique generated names}}{\text{Total generated names}}$$

Interpretation: Measures variation in generated outputs.

Results

Model	Novelty Rate	Diversity
Vanilla RNN	1.0	1.0
BLSTM	1.0	0.995
Attention RNN	1.0	0.510

TASK 3: QUALITATIVE ANALYSIS

Generated Samples

1. Vanilla RNN:

- paliraanthi
- ushinathian
- shapalirath
- vedraanthaa
- taushadeepa

2. BLSTM:

- Pravrantayk
- Pranay
- shavranitha
- Garvita
- arayan

3. Attention RNN:

- (add 5–10 names)

Observations

Vanilla RNN:

- Often produces repetitive or less structured names
- Struggles with long dependencies

BLSTM:

- Produces more realistic names
- Better captures structure due to bidirectional context

Attention RNN:

- Generates more coherent names
- Sometimes overfits patterns or produces unusual combinations

Failure Modes

- Repetition of characters
- Unnatural combinations of syllables
- Very short or very long names

FINAL COMPARISON

- Vanilla RNN is simplest but least effective
- BLSTM improves realism significantly
- Attention model provides best contextual understanding

CONCLUSION

The study shows that increasing model complexity improves performance in character-level generation tasks. While Vanilla RNN captures basic patterns, BLSTM and Attention models generate more realistic and diverse names.